

État actuel

Dans toutes les branches :

1. Paramètre `split_temp` (`bool=False`) défini dans le script de lancement des modèles et injecté dans `settings` avec `BV.settings.update_split_temporal(split_temp)`
2. Transmis à MODFLOW lors de l'appel de `preprocessing_modflow()` (cf `watershed_root.py`)
3. Utilisé L323 de `modflow.py` :

1. si `split_temp` est `True` :
 - a) si la recharge est une `pandas.Series` :
 - si l'index de cette série est constitué de dates :
 - `perlen` est calculé à partir des dates en supposant que l'unité temporelle du modèle est le jour
 - sinon :
 - `perlen` est calculé à partir des indices (ex : si les indices augmentent de 10 en 10, alors `perlen` = 10)
 - b) sinon : cas non défini
2. si `split_temp` est une liste :
 - a) `perlen` = `split_temp`
3. si `split_temp` est `False` :
 - a) `perlen` = 1 partout

Objectifs

Voilà en gros ce qui était ressorti de nos discussions au sujet de ce qu'on attend de cette fonctionnalité, si je n'ai rien oublié ?

- ☐ Permettre différentes possibilités
 - Possibilité de calcul automatique du `perlen`
 - Possibilité d'avoir des `perlen` irréguliers
 - Possibilité de définir le `perlen` par l'utilisateur
- ☐ Clarifier et homogénéiser les notations
 - Renommer `split_temp` en `dis_perlen` voire `perlen`
 - Adapter le paramètre `itmoni` de MODFLOW qui écrit l'unité de temps dans les outputs
- ☐ Prévenir les erreurs des utilisateurs
 - Mettre une info pour prévenir l'utilisateur sur les unités de temps
 - Mettre un warning sur un bilan de recharge, si le bilan de recharge a une tête trop élevée
 - `dis_perlen` = 'D' par défaut ?

Proposition d'implémentation

Prérequis

Quand on en avait discuté j'avais noté la possibilité d'avoir des unités de temps différentes entre la recharge et les paramètres (K). Finalement avec le recul j'ai du mal à voir dans quel cas cela serait utile ? Je me dis qu'il serait plutôt préférable que les unités de temps soient toujours homogènes

dans les inputs rentrés par l'utilisateur, et qu'il vaudrait mieux considérer ça comme une règle imposée. D'ailleurs à ma connaissance c'est également présenté dans MODFLOW comme une règle :

ITMUNI—indicates the time unit of model data, which must be consistent for all data values that involve time. For example, if years is the chosen time unit, then stress-period length, time-step length, transmissivity, and so on, must all be expressed using years for their time units. Note that the program will still run even if “undefined” time units are specified because the fundamental equations used in MODFLOW do not require that the time unit be identified; but be sure to use consistent units for all input data even when ITMUNI indicates an undefined time unit. When the time unit is defined, MODFLOW uses it to print a table of elapsed simulation time:

0 - undefined	3 - hours
1 - seconds	4 - days
2 - minutes	5 - years

[source: [MODFLOW-2005](#)]

Vous en pensez quoi ?

À part ça, d'un point de vue de l'organisation du code, ça me semblait logique de tout gérer au niveau de la recharge, plutôt qu'au niveau des settings. Qu'est-ce que vous en pensez ?

Proposition de modifications

Je vous propose du coup :

1. De faire apparaître cette consigne d'homogénéité des unités de temps dans le **ReadTheDoc** :
 - Au passage je pense que ça serait bien de faire une page « Run a first simulation » qui détaille pas-à-pas toutes les sections et étapes d'un exemple typique, et dans laquelle on pourrait glisser ce genre de recommandation.
 - J'avais fait une doc de ce genre pour l'exemple lakere, je pourrais la recycler dans le ReadTheDoc comme base pour cette page « Run a first simulation ».
2. De faire apparaître cette consigne d'homogénéité des unités de temps également en **commentaire** dans les exemples :
 - Au début de la section `# RECHARGE`, mettre un commentaire du style : `# Be sure to use consistent time and length units for all input data (climatic timeseries, parameters...)`
3. De faire apparaître un **message d'information** lorsque l'utilisateur spécifie le `freq`, `freq_input`, ou `time_step` dans l'appel des fonctions climatiques :
 - Un message du style : `<freq> has been set to .... Note that the time unit of climatic input is independant from <freq> and should be consistent for all input data.`

en lien avec le point précédent :

4. D'homogénéiser le **nom** de ce paramètre `freq` (dans `update_recharge_synthetic`), `time_step` (dans `update_recharge_explore1`), et `freq_input` (dans `update_sim2_reanalysis`), par soucis de clarté
 - Vu qu'il est question ici de l'échantillonnage temporel des inputs, et pour ne pas confondre avec la notion de `time_step` de MODFLOW, je propose de garder `freq`
 - d'ailleurs dans `climatic.py`, on a `self.freq = time_step`, donc autant remplacer dès le départ `time_step` par `freq`
 - en plus `freq` sera cohérent avec la notation pandas

en lien avec le point précédent :

5. Pour éviter toute confusion, je propose de **renommer** le paramètre `freq_time` qui est parfois utilisé pour sous-échantillonner les sorties timeseries, en `freq_postproc`, et de ne pas le définir dans le paragraphe `# Climatic settings` mais plutôt dans un paragraphe `# Post-processing`

6. De rajouter un **garde-fou** comme préconisé par Alexandre, dans `modflow.py` :
 - Si la recharge semble trop élevée par rapport aux valeurs des paramètres, un warning s'affiche, du style : `climatic inputs seem abnormally high/low compared to parameters. Be sure to use consistent units for all input data`
 - C'est pas évident de fixer une règle car K peut varier beaucoup plus que les unités de la recharge, mais on peut essayer
 - En partant du principe que **K** est généralement entre $1e^{-8}$ et $1e^{-3}$ m/s, et que la recharge annuelle est généralement entre $2.4 \cdot 10^{-6}$ et $3.2 \cdot 10^{-5}$ m/s, on peut afficher un message d'avertissement si $K/moy(R) < 3.125e^{-4}$ ou > 417
 - On peut aussi faire la même chose sur l'**épaisseur** en considérant que l'épaisseur est généralement comprise entre 10 et 300 m (est-ce que HMP est prévu pour être utilisé aussi au-delà ?), et donc si $thick/moy(R) < 3.125e^{-5}$ ou $> 1.25e^8$, on affiche un message d'avertissement
7. De renommer le paramètre `split_temp` en `dis_perlen`, ou juste `perlen`, et de le déplacer de la section `# UPDATE` vers la section `# RECHARGE`
 - Dans ce cas il conviendrait aussi de le gérer dans la classe `climatic` et pas `settings`
8. Et enfin, de gérer ce paramètre `dis_perlen` de la manière suivante :

1. Si l'utilisateur veut qu'HMP déduise lui-même le `perlen` à partir de la recharge, alors il doit lui spécifier l'**unité temporelle** du modèle : `dis_perlen = 'D' | 'W' | 'M' | 'min' | 'ms' |` etc. (NB : ajouter en commentaire un lien vers les [alias de fréquence](#) de pandas)
 - a) Si la recharge est une unique valeur ou une simple **liste** de valeurs, alors on affiche un avertissement : `Recharge has no datetime index, stress period were assumed to last 1 <dis_perlen>` et on postule alors que chaque période dure 1 unité de temps : `self.perlen = np.ones(len(self.climatic))`
 - b) Si la chronique est une **série pandas**, alors on n'a pas besoin de postuler, on peut obtenir la durée de chaque période grâce à l'index :
 - Si les indices sont des **integers**, alors on affiche un message d'avertissement : `Recharge has no datetime index, stress period length was based on recharge index numeric values`, et on calcule la durée de chaque période : `self.perlen = self.climatic.index.to_series().diff().values` (ex : si les indices augmentent de 10 en 10, alors `self.perlen = [10, 10, 10...]`)
 - Si les indices sont des **datetime**, alors on peut obtenir sans aucun postulat la durée de chaque période : `self.perlen = self.climatic.index.to_series().diff().dt.total_seconds().values/<time_unit_conv>` (`<time_unit_conv> = 86400` si `dis_perlen == 'D'`, etc.)
2. Si l'utilisateur ne veut pas qu'HMP déduise lui-même le `perlen`, alors il peut spécifier :
 - a) `dis_perlen = [..., ..., ...]` (list) ou `dis_perlen = ...` (unique valeur)
→ `self.perlen = self.dis_perlen`
 - b) `dis_perlen = False`
→ `self.perlen = np.ones(len(self.climatic))` (1 partout)

Dans le cas **1.** on peut également adapter le paramètre `itmuni` dans `modflow.py` pour qu'il reflète l'unité de temps définie par l'utilisateur (ça ne joue pas sur les calculs, c'est juste plus propre).

Dans tous les cas, la 1^{re} valeur de `perlen` sera fixée égale à 1 comme la 1^{re} période est toujours en steady-state.